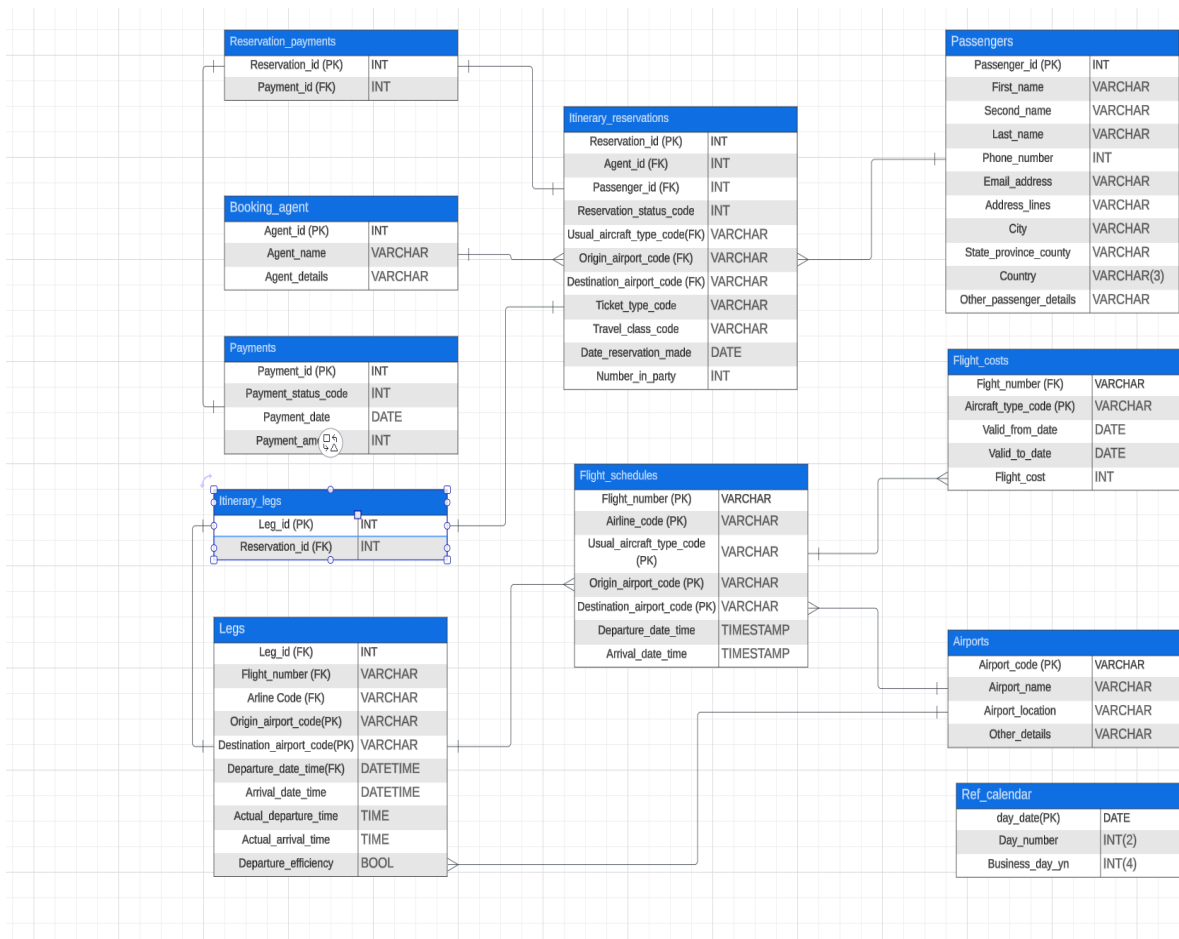


Barrett Viator
DSE6210
Professor Lowhorn
September 29, 2024

Project 1

Part 1 – Data Model

The proposed architectural design of the database included some foreign keys that did not have corresponding primary keys in other tables and has been modified to properly join data efficiently across tables. Additional information from existing tables was deemed necessary to add to the Itinerary view to provide users with information about their flight that is more standard with their experience with other airlines. A Boolean value was also added for employees to be able to easily see if a flight departed on time. These modifications from the original design should enhance both performance and user satisfaction.



Part 2 – Implementation

✓ Schemas (7)	9
✓ airline	10
> Aggregates	11
> Collations	12
> Domains	13
> FTS Configurations	14
> FTS Dictionaries	15
> FTS Parsers	16
> FTS Templates	17
> Foreign Tables	18
> Functions	19
> Materialized Views	20
> Operators	21
> Procedures	22
> Sequences	23
✓ Tables (11)	24
> airports	25
> booking_agents	26
> flight_costs	27
> flight_schedules	28
> itinerary_legs	29
> itinerary_reservations	30
> legs	31
> passengers	32
> payments	33
> ref_calendar	34
> reservation_payments	35
> Trigger Functions	36
> Types	37
> Views	38

Query Query History

```

1 CREATE SCHEMA IF NOT EXISTS airline;
2
3 CREATE TABLE IF NOT EXISTS airline.booking_agents (
4     Agent_id INT NOT NULL,
5     Agent_name VARCHAR NOT NULL,
6     Agent_details VARCHAR NOT NULL,
7     PRIMARY KEY (Agent_id)
8 );
9
10 CREATE TABLE IF NOT EXISTS airline.payments (
11     Payment_id INT NOT NULL,
12     Payment_status_code INT NOT NULL,
13     Payment_date DATE NOT NULL,
14     Payment_amount INT NOT NULL,
15     PRIMARY KEY (Payment_id)
16 );
17
18
19 CREATE TABLE IF NOT EXISTS airline.reservation_payments (
20     Reservation_id INT NOT NULL,
21     Payment_id INT NOT NULL,
22     PRIMARY KEY (Reservation_id),
23     FOREIGN KEY (Payment_id) REFERENCES airline.payments(Payment_id)
24     ON UPDATE CASCADE ON DELETE CASCADE
25 );
26
27 CREATE TABLE IF NOT EXISTS airline.passengers (
28     Passenger_id INT NOT NULL,
29     First_name VARCHAR NOT NULL,
30     Second_name VARCHAR NOT NULL,
31     Last_name VARCHAR NOT NULL,
32     Phone_number INT NOT NULL,
33     Email_address VARCHAR NOT NULL,
34     Address_lines VARCHAR NOT NULL,
35     City VARCHAR NOT NULL,
36     State_province_county VARCHAR NOT NULL,
37     Country VARCHAR(3) NOT NULL,
38     Other_passenger_details VARCHAR,
39     PRIMARY KEY (Passenger_id)
40 );
41
42 CREATE TABLE IF NOT EXISTS airline.flight_schedules (
43     Flight_number VARCHAR NOT NULL,
44     Airline_code VARCHAR NOT NULL,
45     Usual_aircraft_type_code VARCHAR NOT NULL,
46     Origin_airport_code VARCHAR NOT NULL,
47     Destination_airport_code VARCHAR NOT NULL,
48     Departure_date_time TIMESTAMP NOT NULL,
49     Arrival_date_time TIMESTAMP NOT NULL,
50     PRIMARY KEY (Flight_number, Airline_code, Usual_aircraft_type_code, Origin_airport_code, Destination_airport_code, Departure_date_time)
51 );
52
53 ALTER TABLE airline.flight_schedules
54 ADD CONSTRAINT Flight_number UNIQUE (Flight_number);
55
56 CREATE TABLE IF NOT EXISTS airline.itinerary_legs (
57     Leg_id INT NOT NULL,
58     Reservation_id INT NOT NULL,
59     PRIMARY KEY (Leg_id),
60     FOREIGN KEY (Reservation_id) REFERENCES airline.reservation_payments(Reservation_id)
61     ON UPDATE CASCADE ON DELETE CASCADE
62 );
63
64 ALTER TABLE airline.flight_schedules
65 ADD CONSTRAINT unique_usual_aircraft_type UNIQUE (Usual_aircraft_type_code);
66
67 ALTER TABLE airline.flight_schedules
68 ADD CONSTRAINT origin_airport_code UNIQUE (Origin_airport_code);
69
70 ALTER TABLE airline.flight_schedules
71 ADD CONSTRAINT Destination_airport_code UNIQUE (Destination_airport_code);
72
73

```

```

73 CREATE TABLE IF NOT EXISTS airline.legs (
74     Leg_id INT NOT NULL,
75     Flight_number VARCHAR NOT NULL,
76     Airline_code VARCHAR NOT NULL,
77     Usual_aircraft_type_code VARCHAR NOT NULL,
78     Origin_airport_code VARCHAR NOT NULL,
79     Destination_airport_code VARCHAR NOT NULL,
80     Departure_date_time TIMESTAMP NOT NULL,
81     Arrival_date_time TIMESTAMP NOT NULL,
82     Actual_departure_time TIMESTAMP NOT NULL,
83     Actual_arrival_time TIMESTAMP NOT NULL,
84     Departure_efficiency BOOL,
85     FOREIGN KEY (Leg_id) REFERENCES airline.itinerary_legs(Leg_id),
86     FOREIGN KEY (Flight_number, Airline_code, Usual_aircraft_type_code, Origin_airport_code, Destination_airport_code, Departure_date_time)
87     REFERENCES airline.flight_schedules(Flight_number, Airline_code, Usual_aircraft_type_code, Origin_airport_code, Destination_airport_code, Departure_date_time)
88     ON UPDATE CASCADE ON DELETE CASCADE
89 );
90
91 CREATE TABLE IF NOT EXISTS airline.itinerary_legs (
92     Leg_id INT NOT NULL,
93     Reservation_id VARCHAR NOT NULL,
94     PRIMARY KEY (Leg_id),
95     FOREIGN KEY (Reservation_id) REFERENCES airline.itinerary_reservations(Reservation_id)
96     ON UPDATE CASCADE ON DELETE CASCADE
97 );
98
99 CREATE TABLE IF NOT EXISTS airline.itinerary_reservations(
100     Reservation_id INT NOT NULL,
101     Agent_id INT NOT NULL,
102     Passenger_id INT NOT NULL,
103     Reservation_status_code VARCHAR NOT NULL,
104     Usual_aircraft_type_code VARCHAR NOT NULL,
105     Origin_airport_code VARCHAR NOT NULL,
106     Destination_airport_code VARCHAR NOT NULL,
107     Ticket_type_code VARCHAR NOT NULL,
108     Travel_class_code VARCHAR NOT NULL,
109     Date_reservation_made DATE NOT NULL,
110     Number_in_party INT NOT NULL,
111     PRIMARY KEY (Reservation_id),
112     FOREIGN KEY (Agent_id) REFERENCES airline.booking_agents(Agent_id),
113     FOREIGN KEY (Passenger_id) REFERENCES airline.passengers(Passenger_id),
114     FOREIGN KEY (Usual_aircraft_type_code) REFERENCES airline.flight_schedules(Usual_aircraft_type_code),
115     FOREIGN KEY (Origin_airport_code) REFERENCES airline.flight_schedules(Origin_airport_code),
116     FOREIGN KEY (Destination_airport_code) REFERENCES airline.flight_schedules(Destination_airport_code)
117     ON UPDATE CASCADE ON DELETE CASCADE
118 );
119
120 CREATE TABLE IF NOT EXISTS airline.airports (
121     Airport_code VARCHAR(3) NOT NULL,
122     Airport_name VARCHAR NOT NULL,
123     Airport_location VARCHAR NOT NULL,
124     Other_details VARCHAR,
125     PRIMARY KEY (Airport_code)
126 );
127
128 CREATE TABLE IF NOT EXISTS airline.ref_calendar (
129     Day_date INT NOT NULL,
130     Day_number INT NOT NULL,
131     Business_day_yn INT NOT NULL,
132     PRIMARY KEY (Day_date)
133 );
134
135 CREATE TABLE IF NOT EXISTS airline.flight_costs (
136     Flight_number VARCHAR NOT NULL,
137     Aircraft_type_code VARCHAR NOT NULL,
138     Valid_from_date DATE NOT NULL,
139     Valid_to_date DATE NOT NULL,
140     Flight_cost INT NOT NULL,
141     PRIMARY KEY (Aircraft_type_code),
142     FOREIGN KEY (Flight_number) REFERENCES airline.flight_schedules(Flight_number)
143 );
144
145 INSERT INTO airline.booking_agents (agent_id, agent_name, agent_details)
146 VALUES (045231, 'Sue Jones', 'Associate '),
147         (067456, 'Brad Collins', 'VP'),
148         (036524, 'Ann Rice', 'Associate')
149 ;
150
151 INSERT INTO airline.payments (payment_id, payment_status_code, payment_date, payment_amount)
152 VALUES (6345, 05, '10-12-2023', 982),
153        (3267, 05, '02-04-2024', 473),
154        (3585, 05, '05-11-2024', 564),
155        (3563, 05, '08-06-2023', 638),
156        (2573, 05, '09-05-2024', 342)
157 ;
158

```

```

158
159 ▾ INSERT INTO airline.reservation_payments (reservation_id, payment_id)
160 VALUES (15457568, 6345),
161 (24668346, 3267),
162 (35635773, 3585),
163 (36725613, 3563),
164 (74677424, 2573)
165 ;
166
167 ▾ INSERT INTO airline.passengers (passenger_id, first_name, second_name, last_name, phone_number, email_address, address_lines, city, state_province_county, country, c
168 VALUES (7463826, 'Mila', 'Dina', 'Romanova', 19287256, 'mdr@live.com', '2782 1st St', 'New York', 'NY', 'USA', 'Member'),
169 (5885922, 'Laura', 'Kelly', 'Juarez', 17389390, 'lkj25@gmail.com', '1080 Tree Cove', 'Dallas', 'TX', 'USA', 'Gold Status'),
170 (9284293, 'Frank', 'Will', 'Craft', 18373845, 'fwk6@apple.com', '25 Smith St', 'Tempe', 'AZ', 'USA', 'guest checkout' ),
171 (7375758, 'Jian', 'Li', 'Cho', 12384658, 'jlc@yahoo.com', '9876 Rue Verdun', 'Montreal', 'Quebec', 'CAN', 'Member'),
172 (2772849, 'Joe', 'Preston', 'Hue', 13458293, 'jph@windows.com', '47 Turnpike Lane', 'Lafayette', 'LA', 'USA', 'Gold Status')
173 ;
174
175 ▾ INSERT INTO airline.flight_schedules (flight_number, airline_code, usual_aircraft_type_code, origin_airport_code, destination_airport_code, departure_date_time, arriv
176 VALUES
177 ('AA321', 'JK35748', 'B747', 'LFT', 'JFK', '2024-01-01 17:00:00', '2024-01-01 20:00:00'),
178 ('AA642', 'JK26747', 'B360', 'MSY', 'DFW', '2024-01-02 14:00:00', '2024-01-02 15:00:00'),
179 ('AA985', 'JK10832', 'S823', 'IAH', 'ATL', '2024-01-03 15:00:00', '2024-01-02 17:00:00')
180 ;
181
182 ▾ INSERT INTO airline.itinerary_legs (reservation_id, leg_id)
183 VALUES
184 (15457568, 56432),
185 (35635773, 656833),
186 (36725613, 345795)
187 ;
188
189 ▾ INSERT INTO airline.legs (leg_id, flight_number, airline_code, usual_aircraft_type_code, origin_airport_code, destination_airport_code,
190 departure_date_time, arrival_date_time, actual_departure_time, actual_arrival_time, departure_efficiency)
191 VALUES
192 (56432, 'AA321', 'JK35748', 'B747', 'LFT', 'JFK', '2024-01-01 17:00:00', '2024-01-01 20:00:00', '2024-01-01 17:30:00', '2024-01-01 20:30:00', FALSE),
193 (656833, 'AA642', 'JK26747', 'B360', 'MSY', 'DFW', '2024-01-02 14:00:00', '2024-01-02 15:00:00', '2024-01-02 14:00:00', '2024-01-02 15:00:00', TRUE),
194 (345795, 'AA985', 'JK10832', 'S823', 'IAH', 'ATL', '2024-01-03 15:00:00', '2024-01-02 17:00:00', '2024-01-03 15:00:00', '2024-01-02 17:00:00', TRUE)
195 ;
196
197
198 ----- Show total sales for a given flight
199 ▾ SELECT SUM(airline.payments.payment_amount) AS sales_total
200 FROM airline.payments
201 JOIN airline.reservation_payments ON airline.payments.Payment_id = airline.reservation_payments.Payment_id
202 JOIN airline.itinerary_reservations ON airline.reservation_payments.Reservation_id = airline.itinerary_reservations.Reservation_id
203 WHERE airline.itinerary_reservations.Usual_aircraft_type_code = 'B360';
204
205 ----- Show if a flight was on time or delayed
206
207 ▾ SELECT
208 Flight_number,
209 Airline_code,
210 Usual_aircraft_type_code,
211 Origin_airport_code,
212 Destination_airport_code,
213 Departure_date_time,
214 Actual_departure_time,
215 Arrival_date_time,
216 Actual_arrival_time,
217 CASE
218 WHEN Actual_departure_time <= Departure_date_time THEN 'On Time'
219 ELSE 'Delayed'
220 END AS Departure_Status,
221 CASE
222 WHEN Actual_arrival_time <= Arrival_date_time THEN 'On Time'
223 ELSE 'Delayed'
224 END AS Arrival_Status
225 FROM
226 airline.legs
227 ORDER BY
228 Flight_number;
229

```

```

230 ----- Show all customers who have a seat on a given flight
231
232 ✓ SELECT
233     airline.passengers.Passenger_id,
234     airline.passengers.First_name,
235     airline.passengers.Second_name,
236     airline.passengers.Last_name
237 FROM
238     airline.passengers
239 JOIN
240     airline.itinerary_reservations ON airline.passengers.Passenger_id = airline.itinerary_reservations.Passenger_id
241 JOIN
242     airline.itinerary_legs ON airline.itinerary_reservations.Reservation_id = airline.itinerary_legs.Reservation_id
243 JOIN
244     airline.legs ON airline.itinerary_legs.Leg_id = airline.legs.Leg_id
245 WHERE
246     airline.legs.Flight_number = 'AA321'
247 ;
248
249 ----- Show all flights for a given airport
250
251 ✓ SELECT
252     airline.flight_schedules.Flight_number,
253     airline.flight_schedules.Airline_code,
254     airline.flight_schedules.Usual_aircraft_type_code,
255     airline.flight_schedules.Origin_airport_code,
256     airline.flight_schedules.Destination_airport_code,
257     airline.flight_schedules.Departure_date_time,
258     airline.flight_schedules.Arrival_date_time
259 FROM
260     airline.flight_schedules
261 WHERE
262     airline.flight_schedules.Origin_airport_code = 'JFK'
263     OR airline.flight_schedules.Destination_airport_code = 'JFK';
264
265 ----- Create a View for Customer and Show Itinerary
266
267 ✓ CREATE VIEW customer_itinerary AS
268 SELECT
269     airline.itinerary_reservations.Reservation_id,
270     airline.passengers.First_name,
271     airline.passengers.Last_name,
272     airline.itinerary_reservations.Agent_id,
273     airline.itinerary_legs.Leg_id,
274     airline.flight_schedules.Flight_number,
275     airline.flight_schedules.Origin_airport_code,
276     airline.flight_schedules.Destination_airport_code,
277     airline.flight_schedules.Departure_date_time,
278     airline.flight_schedules.Arrival_date_time
279 FROM
280     airline.itinerary_reservations
281 JOIN
282     airline.passengers ON airline.itinerary_reservations.Passenger_id = airline.passengers.Passenger_id
283 JOIN
284     airline.itinerary_legs ON airline.itinerary_reservations.Reservation_id = airline.itinerary_legs.Reservation_id
285 JOIN
286     airline.legs ON airline.itinerary_legs.Leg_id = airline.legs.Leg_id
287 JOIN
288     airline.flight_schedules ON airline.legs.Flight_number = airline.flight_schedules.Flight_number
289     AND airline.legs.Airline_code = airline.flight_schedules.Airline_code
290     AND airline.legs.Usual_aircraft_type_code = airline.flight_schedules.Usual_aircraft_type_code
291     AND airline.legs.Origin_airport_code = airline.flight_schedules.Origin_airport_code
292     AND airline.legs.Destination_airport_code = airline.flight_schedules.Destination_airport_code
293     AND airline.legs.Departure_date_time = airline.flight_schedules.Departure_date_time;
294
295 ----- Show Flight Schedules
296
297
298 ✓ SELECT
299     Flight_number,
300     Airline_code,
301     Usual_aircraft_type_code,
302     Origin_airport_code,
303     Destination_airport_code,
304     Departure_date_time,
305     Arrival_date_time
306 FROM
307     airline.flight_schedules
308 ORDER BY
309     Departure_date_time;
310
311

```

Customer Itinerary View

```

261 ----- Create a View for Customer and Show Itinerary
262
263 v CREATE VIEW customer_itinerary AS
264 SELECT
265     airline.itinerary_reservations.Reservation_id,
266     airline.passengers.First_name,
267     airline.passengers.Last_name,
268     airline.itinerary_reservations.Agent_id,
269     airline.itinerary_legs.Leg_id,
270     airline.flight_schedules.Flight_number,
271     airline.flight_schedules.Origin_airport_code,
272     airline.flight_schedules.Destination_airport_code,
273     airline.flight_schedules.Departure_date_time,
274     airline.flight_schedules.Arrival_date_time
275 FROM
276     airline.itinerary_reservations
277 JOIN
278     airline.passengers ON airline.itinerary_reservations.Passenger_id = airline.passengers.Passenger_id
279 JOIN
280     airline.itinerary_legs ON airline.itinerary_reservations.Reservation_id = airline.itinerary_legs.Reservation_id
281 JOIN
282     airline.legs ON airline.itinerary_legs.Leg_id = airline.legs.Leg_id
283 JOIN
284     airline.flight_schedules ON airline.legs.Flight_number = airline.flight_schedules.Flight_number
285     AND airline.legs.Airline_code = airline.flight_schedules.Airline_code
286     AND airline.legs.Usual_aircraft_type_code = airline.flight_schedules.Usual_aircraft_type_code
287     AND airline.legs.Origin_airport_code = airline.flight_schedules.Origin_airport_code
288     AND airline.legs.Destination_airport_code = airline.flight_schedules.Destination_airport_code
289     AND airline.legs.Departure_date_time = airline.flight_schedules.Departure_date_time;
290
291
292
293

```

Data Output Messages Notifications

reservation_id	first_name	last_name	agent_id	leg_id	flight_number	origin_airport_code	destination_airport_code	departure_date_time	arrival_date_time
integer	character varying	character varying	integer	integer	character varying	character varying	character varying	timestamp without time zone	timestamp without time zone

All Customers with seat on given flight

```

291 ----- Select Passengers for a given flight
292
293 v SELECT
294     airline.passengers.Passenger_id,
295     airline.passengers.First_name,
296     airline.passengers.Second_name,
297     airline.passengers.Last_name
298 FROM
299     airline.passengers
300 JOIN
301     airline.itinerary_reservations ON airline.passengers.Passenger_id = airline.itinerary_reservations.Passenger_id
302 JOIN
303     airline.itinerary_legs ON airline.itinerary_reservations.Reservation_id = airline.itinerary_legs.Reservation_id
304 JOIN
305     airline.legs ON airline.itinerary_legs.Leg_id = airline.legs.Leg_id
306 WHERE
307     airline.legs.Flight_number = 'AA321'

```

Data Output Messages Notifications

passenger_id	first_name	second_name	last_name
[PK] integer	character varying	character varying	character varying

All flights for a given airport

```
290 ----- Show all flights for a given airport
291 SELECT
292     airline.flight_schedules.Flight_number,
293     airline.flight_schedules.Airline_code,
294     airline.flight_schedules.Usual_aircraft_type_code,
295     airline.flight_schedules.Origin_airport_code,
296     airline.flight_schedules.Destination_airport_code,
297     airline.flight_schedules.Departure_date_time,
298     airline.flight_schedules.Arrival_date_time
299 FROM
300     airline.flight_schedules
301 WHERE
302     airline.flight_schedules.Origin_airport_code = 'JFK'
303     OR airline.flight_schedules.Destination_airport_code = 'JFK';
304
```

Data Output Messages Notifications

	flight_number [PK] character varying	airline_code [PK] character varying	usual_aircraft_type_code [PK] character varying	origin_airport_code [PK] character varying	destination_airport_code [PK] character varying	departure_date_time [PK] timestamp without time zone	arrival_date_time timestamp without time zone
1	AA321	JK35748	B747	LFT	JFK	2024-01-01 17:00:00	2024-01-01 20:00:00

View flight schedule

```
296 ----- Show Flight Schedules
297
298 SELECT
299     Flight_number,
300     Airline_code,
301     Usual_aircraft_type_code,
302     Origin_airport_code,
303     Destination_airport_code,
304     Departure_date_time,
305     Arrival_date_time
306 FROM
307     airline.flight_schedules
308 ORDER BY
309     Departure_date_time;
310
311
```

Data Output Messages Notifications

	flight_number [PK] character varying	airline_code [PK] character varying	usual_aircraft_type_code [PK] character varying	origin_airport_code [PK] character varying	destination_airport_code [PK] character varying	departure_date_time [PK] timestamp without time zone	arrival_date_time timestamp without time zone
1	AA321	JK35748	B747	LFT	JFK	2024-01-01 17:00:00	2024-01-01 20:00:00
2	AA642	JK26747	B360	MSY	DFW	2024-01-02 14:00:00	2024-01-02 15:00:00
3	AA985	JK10832	S823	IAH	ATL	2024-01-03 15:00:00	2024-01-02 17:00:00

List flights as On-time or Delayed

```
291 ----- Flight On-Time or Delayed
292
293
294 SELECT
295     Flight_number,
296     Airline_code,
297     Usual_aircraft_type_code,
298     Origin_airport_code,
299     Destination_airport_code,
300     Departure_date_time,
301     Actual_departure_time,
302     Arrival_date_time,
303     Actual_arrival_time,
304 CASE
305     WHEN Actual_departure_time <= Departure_date_time THEN 'On Time'
306     ELSE 'Delayed'
307 END AS Departure_status,
308 CASE
309     WHEN Actual_arrival_time <= Arrival_date_time THEN 'On Time'
310     ELSE 'Delayed'
311 END AS Arrival_status
312 FROM
313     airline.flights
314 ORDER BY
315     Flight_number;
316
```

Data Output Messages Notifications

	flight_number character varying	airline_code character varying	usual_aircraft_type_code character varying	origin_airport_code character varying	destination_airport_code character varying	departure_date_time timestamp without time zone	actual_departure_time timestamp without time zone	arrival_date_time timestamp without time zone	actual_arrival_time timestamp without time zone	departure_status text	arrival_status text
1	AA321	JK35748	B747	LFT	JFK	2024-01-01 17:00:00	2024-01-01 17:30:00	2024-01-01 20:00:00	2024-01-01 20:30:00	Delayed	Delayed
2	AA642	JK26747	B360	MSY	DFW	2024-01-02 14:00:00	2024-01-02 14:00:00	2024-01-02 15:00:00	2024-01-02 15:00:00	On Time	On Time
3	AA985	JK10832	S823	IAH	ATL	2024-01-03 15:00:00	2024-01-03 15:00:00	2024-01-02 17:00:00	2024-01-02 17:00:00	On Time	On Time

Calculate total sales of a flight

```

297 ----- Show total sales for a given flight
298
299 |
300 v SELECT SUM(airline.payments.payment_amount) AS sales_total
301 FROM airline.payments
302 JOIN airline.reservation_payments ON airline.payments.Payment_id = airline.reservation_payments.Payment_id
303 JOIN airline.itinerary_reservations ON airline.reservation_payments.Reservation_id = airline.itinerary_reservations.Reservation_id
304 WHERE airline.itinerary_reservations.Usual_aircraft_type_code = 'B360';
305

```

Data Output Messages Notifications

sales_total	bigint
1	[null]

Part 3 – Discussion

The process of creating a database for a highly transactional system such as an airline reservation system involves strategic planning in order to ensure that time and financial resources are not improperly allocated as operations evolve. Considering all factors such as hardware investments, software selection and maintenance, as well as cyber security factors to protect both the organization and the stakeholders is a vital aspect to enable efficiency in both operations and expenditures.

First-normal form has many benefits if this design is initially implemented in a way that does not have to be modified at length as the needs or services of an organization change (Albarak & Bahsoon, 2018, p 32). This form promotes data quality which is an essential part of optimized performance and the reduction of data redundancies is enhanced by using this form. Unfortunately there are many costs incurred via time and fiscal resources if the need to restructure the database to add additional capabilities down the line. Data may need to be migrated in the event that restructuring which can be time intensive, but this process greatly improves the performance (Albarak & Bahsoon, 2018, p 33 -35). For this reason, initially designing the database in first-normal form should be selected, and further modifications should be carefully planned.

Creating Views for customer functions listed in the SRS can help improve the performance of the database by reducing the amount of information needed to be evaluated to return the results of queries.

Segmenting data that is only needed to be accessed by customers also helps alleviate security issues that could more easily arise if views were not created to enforce access controls. Aggregate views increase the ability of data to be further explored with Machine Learning and Artificial Intelligence technologies as well. By designing the airline database to contain views early on in the product life cycle, the airline has the ability to apply more advanced technologies to infer causalities as the organization scales without having to restructure the database further down the line (Youngmann et al., 2024, pp 1 -3)

As operations expand, producing more data impacts the speed at which queries can be performed and executed. A database for an airline is highly transactional and heavily relies on the accuracy of the database due to the nature of how often seats are booked, flights are modified, and how important it is to ensure that the requests made accurately reflect the availability of seats. Query performance can scale efficiently using technologies that promote the benefits of scaling horizontally, but scaling vertically could be an ideal solution for a database such as this one if the resources are available to invest in the necessary hardware.

Dedicated servers owned by the company that utilize newer technologies such as CXL could manage the database in house by distributing the workload to multiple CPUs and reducing the amount of data transfer that would be required to maintain a growing database (Lerner & Alonso, 2024, pp. 2568-2570). Often there are resources lost during the process of transferring data from one server to another even when the database is scaled horizontally. Trends promote horizontal scaling over vertical scaling, but there are options that can make vertical scaling more practical.

CXL adapters can make better use of the bandwidth capabilities of a database system and can pool resources via switches that alleviate issues such as memory not being utilized as efficiently as it could be in current cloud based computing systems (Lerner & Alonso, 2024, pp. 2570-2571). A downside of choosing to scale vertically is not having the data backups stored across different geographic locations in

the event of an environmental disaster. For this reason, a hybrid storage solution may benefit the organization in regards to data recovery strategies.

In both vertical and horizontal scaling, the need to preserve resources for only the most relevant data will arise. Maintaining a schedule to delete data from the database that no longer needs to be stored about customers and reservation histories will also improve the performance of the database by reducing the amount of information needed to be sorted through when executing queries. This is also an important task from a cybersecurity perspective because maintaining personally identifiable information and other protected information is something that must be handled properly. Simply deleting the data and reallocating the storage is not sufficient, but using technologies such as DBSanitizer can aid in the proper deletion of data and storage optimization for future scaling (Wagner & Rasin, 2024, p 1441). By executing a “validated search and destroy approach,” DBSanitizer can more effectively manage the proper deletion of data over the traditional SQL route of “Erase on commit” (Wagner & Rasin, 2024, p 1442).

Properly designing a database before executing the actual process and moving to operating status can greatly benefit a business and its stakeholders. Without carefully evaluating multiple strategies, time and resources can be unnecessarily wasted. For this reason, all options should be explored at great lengths.

References

- Albarak, M., & Bahsoon, R. (2018). Prioritizing technical debt in database normalization using portfolio theory and Data Quality Metrics. *Proceedings of the 2018 International Conference on Technical Debt*, 31–40. <https://doi.org/10.1145/3194164.3194170>
- Lerner, A., & Alonso, G. (2024). CXL and the return of scale-up database engines. *Proceedings of the VLDB Endowment*, 17(10), 2568–2575. <https://doi.org/10.14778/3675034.3675047>

Wagner, J., & Rasin, A. (2024). Forget about it: Batched database sanitization. Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, 14, 1441–1452.
<https://doi.org/10.1145/3605098.3636054>

Youngmann, B., Cafarella, M., Gilad, A., & Roy, S. (2024). Summarized causal explanations for aggregate views. Proceedings of the ACM on Management of Data, 2(1), 1–27. <https://doi.org/10.1145/3639328>